

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 — Ağ Tabanlı Paralel Dağıtım Sistemleri
Proje 6: Veritabanı Yükseltme ve Sürüm Yönetimi

Selim Bedirhan Öztürk
20291277

GitHub: <https://github.com/selimbedirhan/BLM4522-Ag-Tabanli-Paralel-Dagitim-Sistemleri>

ÖZET

Bu projede PostgreSQL 14 üzerinde bir Kütüphane Yönetim Sistemi veritabanı tasarlanmış ve bu veritabanı üzerinde kontrollü sürüm yükseltme (schema migration) altyapısı kurulmuştur. Veritabanı üç aşamalı bir evrim sürecinden geçirilmiştir: v1.0 (temel kütüphane sistemi), v2.0 (genişletilmiş özellikler) ve v3.0 (tam özellikli dijital kütüphane). Her sürüm geçişinde atomik transaction blokları kullanılmış, yükseltme öncesi 7 aşamalı hazırlık kontrolü ve yükseltme sonrası 5 kategoride 28 uyumluluk testi uygulanmıştır. Ayrıca her migration dosyasına karşılık gelen bir rollback mekanizması geliştirilmiş ve sürüm geçmişi bir denetim tablosu aracılığıyla veritabanı içinde izlenmiştir.

İÇİNDEKİLER

1. Giriş ve Motivasyon
2. Ortam ve Kullanılan Teknolojiler
3. Sürüm Yönetimi Mimarisi
4. v1.0 — Temel Kütüphane Sistemi
5. v2.0 — Genişletilmiş Kütüphane Sistemi
6. v3.0 — Tam Özellikli Dijital Kütüphane
7. Yükseltme Öncesi Kontrol Sistemi
8. Migration Yönetici Scripti
9. Rollback Mekanizması
10. Yükseltme Sonrası Uyumluluk Testleri
11. Tam Demo Senaryosu
12. Sürüm Karşılaştırma Analizi
13. Sonuç ve Değerlendirme
14. Kaynakça

1. Giriş ve Motivasyon

Yazılım projeleri sürekli gelişim halindedir ve iş gereksinimleri zamanla değişir. Bu değişikliklerin veritabanı şemasına yansıtılması, yani veritabanının güvenli ve kontrollü bir şekilde yükseltilmesi, kurumsal sistemlerin en zorlayıcı operasyonlarından biridir. Yetersiz bir sürüm yönetimi stratejisi veri kaybına, uygulama kesintilerine ve tutarsız durumlara yol açabilir.

Bu projenin temel amacı, veritabanı sürüm yükseltme süreçlerini sistematik bir şekilde planlamak ve uygulamak, her aşamada oluşabilecek riskleri minimize edecek güvenlik mekanizmaları kurmaktır.

Proje kapsamında ele alınan konular şunlardır:

- Schema migration kavramı ve uygulama yöntemleri
- Sürüm takip tablosu ile migration geçmişinin izlenmesi
- Atomik transaction blokları içinde güvenli DDL/DML işlemleri
- Yükseltme öncesi ön koşul ve bütünlük kontrolleri
- Her migration için eşleşen rollback mekanizması
- Otomatik sürüm tespiti ile zincirli migration uygulanması
- 28 uyumluluk testi ile yükseltme doğrulaması

2. Ortam ve Kullanılan Teknolojiler

2.1 Kullanılan Araçlar

Araç / Teknoloji	Türü	Açıklama
PostgreSQL 14	VTYS	Transactional DDL destekleyen açık kaynak veritabanı sistemi
psql	SQL İstemcisi	Komut satırından SQL çalıştırma ve migration uygulaması
Scripting	Otomasyon	Migration yönetici, kontrol ve test scriptleri
pg_dump	Yedekleme	Yükseltme öncesi güvenlik yedeği
Transaction (BEGIN/COMMIT)	DDL Güvenliği	Atomik migration işlemleri

Araç / Teknoloji	Türü	Açıklama
information_schema	Metadata	Tablo, kolon, constraint varlık kontrolleri
pg_stat_user_tables	İstatistik	Kayıt sayıları ve performans metrikleri

2.2 PostgreSQL'in Transactional DDL Avantajı

Bu projede PostgreSQL tercih edilmesinin en önemli nedenlerinden biri, **DDL komutlarını transaction blokları içinde çalıştırabilme yeteneğidir**. MySQL, Oracle gibi birçok veritabanı sisteminde CREATE TABLE veya ALTER TABLE gibi DDL komutları implicit commit yapar ve geri alınamaz. PostgreSQL'de ise:

```
203 BEGIN;
204     CREATE TABLE yeni_tablo (...);
205     ALTER TABLE mevcut_tablo ADD COLUMN yeni_kolon VARCHAR
206         (50);
207     INSERT INTO schema_version (...) VALUES (...);
208 COMMIT;
```

Bu blok içindeki herhangi bir komutta hata oluşursa tüm değişiklikler otomatik olarak geri alınır (ROLLBACK). Bu özellik, migration işlemlerinin güvenliğini önemli ölçüde artırır.

2.3 Proje Dizin Yapısı

```
Proje6-Surum-Yonetimi
├── backups
├── config
├── logs
├── migrations
│   ├── v1_to_v2_migration.sql
│   └── v2_to_v3_migration.sql
├── rollback
│   ├── v2_to_v1_rollback.sql
│   └── v3_to_v2_rollback.sql
├── scripts
│   ├── 01_v1_temel_sema.sql
│   ├── 02_v1_veri_yukle.sql
│   ├── 03_yukseltme_onesi_kontrol.sh
│   ├── 04_migration_yonetici.sh
│   ├── 05_yukseltme_sonrasi_kontrol.sh
│   └── 06_tam_demo.sh
├── .gitignore
└── README.md
```

3. Sürüm Yönetimi Mimarisi

3.1 schema_version Tablosu

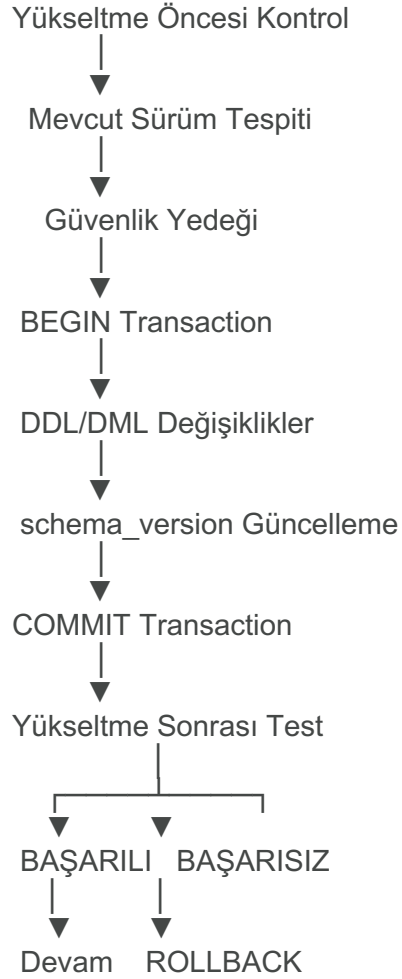
Tüm sürüm yönetimi sürecinin çekirdeğini oluşturan schema_version tablosu, her migration işleminin detaylı kaydını tutar. Bu yaklaşım, Flyway ve Liquibase gibi endüstri standardı araçlardan esinlenilmiştir.

```
17 CREATE TABLE schema_version (  
18     version_id SERIAL PRIMARY KEY,  
19     version_no VARCHAR(20) NOT NULL,  
20     aciklama TEXT,  
21     migration_dosya VARCHAR(200),  
22     uygulama_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
23     uygulayan VARCHAR(100) DEFAULT CURRENT_USER,  
24     sure_ms INTEGER,  
25     durum VARCHAR(20) DEFAULT 'basarili'  
26     CHECK (durum IN ('basarili', 'basarisiz',  
27         'geri_alindi')),  
28     kontrol_toplami VARCHAR(64),  
29     geri_alma_dosya VARCHAR(200)  
30 );
```

Kolon	Tip	Açıklama
version_id	SERIAL	Benzersiz migration numarası
version_no	VARCHAR(20)	Sürüm etiketi (1.0.0, 2.0.0, 3.0.0)
aciklama	TEXT	Migration'ın ne yaptığını açıklayan metin
migration_dosya	VARCHAR(100)	Uygulanan SQL dosyasının adı
uygulama_tarihi	TIMESTAMP	Migration'ın uygulandığı tarih ve saat
sure_ms	INTEGER	İşlem süresi (milisaniye)
durum	VARCHAR(20)	basarili / basarisiz / geri_alindi
geri_alma_dosya	VARCHAR(100)	İlişkili rollback dosyasının adı

3.2 Migration Yaşam Döngüsü

Her sürüm geçişi aşağıdaki yaşam döngüsünü izler:



4. v1.0 — Temel Kütüphane Sistemi

4.1 Veritabanı Oluşturma

Kütüphane veritabanı UTF-8 karakter kodlaması ile oluşturulmuştur:

```
8 DROP DATABASE IF EXISTS kutuphane_db;  
9 CREATE DATABASE kutuphane_db WITH ENCODING = 'UTF8'  
   TEMPLATE = template0;
```

4.2 Tablo Yapıları

v1.0 sürümünde temel kütüphane yönetimi için 5 ana tablo ve 1 sürüm takip tablosu oluşturulmuştur:

#	Tablo	Açıklama	Kayıt	Önemli Alanlar
1	schema_version	Sürüm takip	1	version_no, durum, sure_ms
2	yazarlar	Yazar bilgileri	40	ad, soyad, dogum_tarihi, ulke
3	kategoriler	Kitap kategorileri	12	kategori_adi, aciklama
4	kitaplar	Kitap kataloğu	200	isbn (UNIQUE), baslik, yazar_id, kopya_sayisi
5	uyeler	Üye kayıtları	400	tc_kimlik (UNIQUE), email (UNIQUE), telefon
6	odunc_islemleri	Ödünç alma	2.000	kitap_id, uye_id, durum (CHECK)
	TOPLAM		~2.650	

4.3 Kısıtlamalar ve İlişkiler

v1.0 şemasında tanımlanan kısıtlamalar:

- **5 Foreign Key:** kitaplar→yazarlar, kitaplar→kategoriler, odunc→kitaplar, odunc→uyeler
- **CHECK Constraints:** odunc_islemleri.durum IN ('odunccte', 'iade_edildi', 'gecikti')
- **UNIQUE Constraints:** isbn, tc_kimlik, email
- **NOT NULL:** Tüm kritik alanlarda zorunluluk

4.4 İndeksler

```
97 CREATE INDEX idx_kitaplar_isbn ON kitaplar(isbn);
98 CREATE INDEX idx_kitaplar_yazar ON kitaplar(yazar_id);
99 CREATE INDEX idx_kitaplar_kategori ON kitaplar(kategori_id);
100 CREATE INDEX idx_uyeler_tc ON uyeler(tc_kimlik);
101 CREATE INDEX idx_odunc_kitap ON odunc_islemleri(kitap_id);
102 CREATE INDEX idx_odunc_uye ON odunc_islemleri(uye_id);
103 CREATE INDEX idx_odunc_durum ON odunc_islemleri(durum);
```

4.5 Örnek Veri

Veri üretiminde PostgreSQL'in `generate_series()` fonksiyonu kullanılmış, Türkçe isim dizileri (Orhan, Yaşar, Elif...) ve gerçekçi kitap başlıkları (Beyaz Kale, İnce Memed, Tutunamayanlar...) ile doğal görünümlü kayıtlar oluşturulmuştur. Ödünç işlemlerinde `random()` fonksiyonu ile son 1 yıla yayılan tarihler ve farklı durumlar (oduncu, iade_edildi, gecikti) atanmıştır.

5. v2.0 — Genişletilmiş Kütüphane Sistemi

v1.0 → v2.0 migration'ı, kütüphane sisteminin işlevselliğini önemli ölçüde genişletir.

5.1 Migration Koruma Mekanizması

Migration dosyasının başında mevcut sürümün doğrulanması sağlanır:

```
152 DO $$
153 BEGIN
154     IF NOT EXISTS (
155         SELECT 1 FROM schema_version
156         WHERE version_no = '1.0.0' AND durum = 'basarili'
157     ) THEN
158         RAISE EXCEPTION 'v1.0 bulunamadı! Önce v1.0
159         kurulmalı.';
160     END IF;
161 END $$;
```

Bu kontrol, migration'ların yanlış sırada uygulanmasını engeller.

5.2 Yeni Tablolar

Tablo	Açıklama	Kayıt	Açıklama
yayinevleri	Yayınevi bilgileri	8	ad, sehir, iletisim_email
kitap_yazarlar	Çoka-çok yazar ilişkisi	200	kitap_id, yazar_id (UNIQUE pair)
cezalar	Gecikme ceza sistemi	~200	odunc_id, ceza_tutari, odendi
rezervasyonlar	Kitap rezervasyonu	300	kitap_id, uye_id, durum, bitis_tarihi

5.3 Kolon Eklemeleri (ALTER TABLE)

Mevcut tablolara yeni kolonlar eklenir:

```
72 ALTER TABLE kitaplar ADD COLUMN yayinevi_id INTEGER
   REFERENCES yayinevleri(yayinevi_id);
73 ALTER TABLE kitaplar ADD COLUMN dil VARCHAR(30) DEFAULT
   'Turkce';
74 ALTER TABLE kitaplar ADD COLUMN aciklama TEXT;
75 ALTER TABLE kitaplar ADD COLUMN etiketler VARCHAR(500);
76
77 ALTER TABLE uyeler ADD COLUMN uyelik_tipi VARCHAR(20)
   DEFAULT 'standart'
78     CHECK (uyelik_tipi IN ('standart', 'ogrenci',
   'akademik', 'vip'));
79 ALTER TABLE uyeler ADD COLUMN dogum_tarihi DATE;
80 ALTER TABLE uyeler ADD COLUMN max_odunc INTEGER DEFAULT 3;
81
82 ALTER TABLE yazarlar ADD COLUMN biyografi TEXT;
83
84 ALTER TABLE odunc_islemleri ADD COLUMN notlar TEXT;
85 ALTER TABLE odunc_islemleri ADD COLUMN uzatma_sayisi
   INTEGER DEFAULT 0;
```

5.4 Veri Migrasyonu

Yeni eklenen tablolar ve kolonlar, mevcut verilerle tutarlı olacak şekilde doldurulur:

```
157 UPDATE kitaplar SET yayinevi_id = 1 + (kitap_id % 8);
158
159
160 UPDATE uyeler SET uyelik_tipi = (ARRAY['standart','ogrenci',
   'akademik','vip'])[1 + (uye_id % 4)],
161     max_odunc = CASE
162     WHEN (ARRAY['standart','ogrenci','akademik','vip'])
   [1 + (uye_id % 4)] = 'vip' THEN 10
163     WHEN (ARRAY['standart','ogrenci','akademik','vip'])
   [1 + (uye_id % 4)] = 'akademik' THEN 7
164     WHEN (ARRAY['standart','ogrenci','akademik','vip'])
   [1 + (uye_id % 4)] = 'ogrenci' THEN 5
165     ELSE 3 END;
```

5.5 Yeni Veritabanı Objeleri

- **2 View:** v_kitap_detay (kitap-yazar-yayınevi bilgisi), v_uye_ceza_durumu (üye ceza özeti)
- **1 Fonksiyon:** fn_ceza_hesapla(odunc_id) — Gecikme gün sayısına göre ceza tutarını hesaplar
- **7 Yeni İndeks:** yayinevi, ceza, rezervasyon tabloları üzerine performans indeksleri

5.6 Sürüm Kaydı

Migration tamamlandığında schema_version tablosuna kayıt eklenir:

```
183 INSERT INTO schema_version (version_no, aciklama,  
184 migration_dosya, geri_alma_dosya)  
185 VALUES ('2.0.0', 'Yayinevleri, ceza sistemi, rezervasyon,  
kitap-yazar coka-cok, uyelik tipleri',  
'v1_to_v2_migration.sql', 'v2_to_v1_rollback.sql');
```

6. v3.0 — Tam Özellikli Dijital Kütüphane

v2.0 → v3.0 migration'ı, kütüphane sistemini dijital ortama taşıyan kapsamlı bir güncellemedir.

6.1 Yeni Tablolar

Tablo	Açıklama	Kayıt	Önemli Alanlar
dijital_kitaplar	E-kitap ve sesli kitap	80	format (epub/pdf/ses), dosya_boyutu_mb, indirme_sayisi
etkinlikler	Kütüphane etkinlikleri	20	baslik, tur, kontenjan, katilimci_sayisi
etkinlik_katilim	Etkinlik katılım kayıtları	—	etkinlik_id, uye_id (UNIQUE pair)
degerlendirmeler	Kitap puanlama/yorum	~400	puan (1-5 CHECK), yorum
bildirimler	Üye bildirimleri	600	tip, mesaj, okundu

6.2 Kolon Eklemelemleri

```
87 ALTER TABLE kitaplar ADD COLUMN ort_puan DECIMAL(3,2) DEFAULT 0;  
88 ALTER TABLE kitaplar ADD COLUMN degerlendirme_sayisi INTEGER DEFAULT 0;  
89 ALTER TABLE kitaplar ADD COLUMN dijital_mevcut BOOLEAN DEFAULT FALSE;  
90  
91 ALTER TABLE uyeler ADD COLUMN toplam_odunc INTEGER DEFAULT 0;  
92 ALTER TABLE uyeler ADD COLUMN gecikme_sayisi INTEGER DEFAULT 0;  
93 ALTER TABLE uyeler ADD COLUMN puan INTEGER DEFAULT 0;
```

6.3 Trigger Mekanizması

Değerlendirme eklendiğinde ilgili kitabın ortalama puanını otomatik güncelleyen trigger:

```
135 CREATE OR REPLACE FUNCTION fn_puan_guncelle()
136 RETURNS TRIGGER AS $$
137 BEGIN
138     UPDATE kitaplar SET
139         ort_puan = (SELECT ROUND(AVG(puan)::NUMERIC, 2) FROM degerlendirmeler WHERE kitap_id = NEW.kitap_id),
140         degerlendirme_sayisi = (SELECT COUNT(*) FROM degerlendirmeler WHERE kitap_id = NEW.kitap_id)
141     WHERE kitap_id = NEW.kitap_id;
142     RETURN NEW;
143 END;
144 $$ LANGUAGE plpgsql;
145
146 CREATE TRIGGER trg_puan_guncelle
147     AFTER INSERT OR UPDATE ON degerlendirmeler
148     FOR EACH ROW EXECUTE FUNCTION fn_puan_guncelle();
```

6.4 Gelişmiş Görünümler

v_kutuphane_istatistik: Kütüphanenin genel durumunu tek sorguda gösteren dashboard view:

```
108 CREATE VIEW v_kutuphane_istatistik AS
109 SELECT
110     (SELECT COUNT(*) FROM kitaplar) AS toplam_kitap,
111     (SELECT COUNT(*) FROM uyeler) AS toplam_uye,
112     (SELECT COUNT(*) FROM odunc_islemleri WHERE durum='odunc') AS aktif_odunc,
113     (SELECT COUNT(*) FROM dijital_kitaplar) AS dijital_kitap,
114     (SELECT COUNT(*) FROM etkinlikler WHERE tarih > CURRENT_DATE) AS
    yaklasan_etkinlik;
```

v_populer_kitaplar: En çok ödünç alınan ve en yüksek puanlı kitaplar.

7. Yükseltme Öncesi Kontrol Sistemi

Yükseltmeye başlamadan önce 7 aşamalı bir hazırlık kontrolü uygulanır. Her kontrol sonucu OK, FAIL veya UYARI olarak raporlanır.

7.1 Kontrol Aşamaları

#	Kontrol	Açıklama	Yöntem
1	Bağlantı Kontrolü	PostgreSQL sunucusu ve DB erişimi	pg_isready + psql SELECT 1
2	Sürüm Bilgisi	Mevcut sürüm ve migration geçmişi	schema_version sorgusu
3	Veritabanı Durumu	Tablo/indeks/view/fonksiyon sayıları	information_schema, pg_indexes

#	Kontrol	Açıklama	Yöntem
4	Veri Bütünlüğü	FK ihlali, NULL, duplicate kontrol	JOIN + COUNT sorguları
5	Aktif İşlemler	Uzun çalışan sorgu kontrolü	pg_stat_activity
6	Disk Alanı	Yeterli alan kontrolü	df -h
7	Güvenlik Yedeği	Otomatik yükseltme öncesi yedek	pg_dump -Fc

7.2 Veri Bütünlüğü Kontrolleri

Foreign key ilişkilerinin tutarlılığı LEFT JOIN ile doğrulanır:

```

115 FK_VIOLATIONS=$(psql -U "$DB_USER" -d "$DB_NAME" -t -A -c "
116     SELECT COUNT(*) FROM odunc_islemleri oi
117     LEFT JOIN kitaplar k ON oi.kitap_id = k.kitap_id
118     WHERE k.kitap_id IS NULL;
119 " 2>/dev/null)
120 [ "$FK_VIOLATIONS" = "0" ] && check "Foreign key butunlugu" 0 || check "FK ihlali:
    $FK_VIOLATIONS" 1

```

Bu sorgu, kitaplar tablosunda karşılığı olmayan ödünç kayıtlarını (orphan records) tespit eder. Ayrıca ISBN alanında NULL ve duplicate kontrolleri yapılır.

7.3 Sonuç Raporlama

Script, tüm kontrollerin sonucunda bir özet rapor sunar:

```

Başarılı: 8 | Başarısız: 0 | Uyarı: 0
Mevcut sürüm: 1.0.0
Yedek: pre_upgrade_1.0.0_20260413_120000.dump

```

YÜKSELTMEYE HAZIR!

Herhangi bir kontrol başarısız olursa yükseltmeye devam edilmemesi yönünde uyarı verilir.

8. Migration Yönetici Scripti

04_migration_yonetici.sh scripti, mevcut sürümü otomatik tespit ederek gerekli migration'ları zincirleme olarak uygular.

8.1 Otomatik Sürüm Tespiti

```

63 CURRENT_VER=$(psql -U "$DB_USER" -d "$DB_NAME" -t -A -c "
64     SELECT version_no FROM schema_version WHERE durum='basarili' ORDER BY version_id
65     DESC LIMIT 1;
66 " 2>/dev/null)

```

8.2 Zincirli Migration Mantığı

Script, mevcut sürüme göre uygulanması gereken migration'ları belirler ve sırasıyla çalıştırır:

Mevcut Sürüm	Uygulanacak Migration'lar
v1.0.0	v1→v2 → v2→v3
v2.0.0	v2→v3
v3.0.0	Zaten güncel, işlem yok

8.3 Migration Uygulama Süreci

Her migration öncesi yedek alınır, migration SQL dosyası psql -f ile uygulanır ve çıkış kodu kontrol edilir:

```
76 # Yedek al
77 pg_dump -U "$DB_USER" -d "$DB_NAME" -Fc -f "$BACKUP_FILE"
78
79 # Migration uygula
80 psql -U "$DB_USER" -d postgres -f "$MIGRATION_FILE"
81
82 # Sonuç kontrol
83 if [ $? -eq 0 ]; then
84     echo "Migration başarılı"
85 else
86     echo "Migration BAŞARISIZ - Rollback gerekebilir"
87 fi
```

9. Rollback Mekanizması

Her migration dosyası için karşılık gelen bir rollback dosyası geliştirilmiştir. Rollback, başarısız bir migration sonrası veritabanını önceki kararlı durumuna geri döndürür.

9.1 Rollback Dosya Eşleştirmesi

Migration	Rollback	Geri Alınan Değişiklikler
v1_to_v2_migration.sql	v2_to_v1_rollback.sql	4 tablo, ~10 kolon, 2 view, 1 fonksiyon
v2_to_v3_migration.sql	v3_to_v2_rollback.sql	5 tablo, ~6 kolon, 2 view, 1 trigger

9.2 Rollback Süreci (v3.0 → v2.0 Örneği)

```
4  \c kutuphane_db;
5  BEGIN;
6
7  DROP TABLE IF EXISTS bildirimler CASCADE;
8  DROP TABLE IF EXISTS etkinlik_katilim CASCADE;
9  DROP TABLE IF EXISTS etkinlikler CASCADE;
10 DROP TABLE IF EXISTS degerlendirmeler CASCADE;
11 DROP TABLE IF EXISTS dijital_kitaplar CASCADE;
12
13 DROP VIEW IF EXISTS v_kutuphane_istatistik;
14 DROP VIEW IF EXISTS v_populer_kitaplar;
15 DROP FUNCTION IF EXISTS fn_puan_guncelle() CASCADE;
16
17 ALTER TABLE kitaplar DROP COLUMN IF EXISTS ort_puan;
18 ALTER TABLE kitaplar DROP COLUMN IF EXISTS degerlendirme_sayisi;
19 ALTER TABLE kitaplar DROP COLUMN IF EXISTS dijital_mevcut;
20 ALTER TABLE uyeler DROP COLUMN IF EXISTS toplam_odunc;
21 ALTER TABLE uyeler DROP COLUMN IF EXISTS gecikme_sayisi;
22 ALTER TABLE uyeler DROP COLUMN IF EXISTS puan;
23
24 UPDATE schema_version SET durum = 'geri_alindi' WHERE version_no = '3.0.0';
25 INSERT INTO schema_version (version_no, aciklama, migration_dosya)
26 VALUES ('2.0.0-rollback', 'v3.0 geri alindi, v2.0 durumuna donuldu',
27         'v3_to_v2_rollback.sql');
28 COMMIT;
29 SELECT 'Rollback v3.0 -> v2.0 BASARILI!' AS bilgi;
```

9.3 Rollback Doğrulaması

Demo scriptinde rollback sonrası silinen tabloların gerçekten kaldırıldığı doğrulanır:

```
89 for tbl in dijital_kitaplar etkinlikler bildirimler; do
90     EXISTS=$(psql -U "$DB_USER" -d "$DB_NAME" -t -A -c "
91         SELECT COUNT(*) FROM information_schema.tables WHERE table_name='$tbl';
92         " 2>/dev/null)
93     if [ "$EXISTS" = "0" ]; then
94         echo -e "  ${GREEN}[OK]${NC} $tbl tablosu silindi"
95     else
96         echo -e "  ${RED}[FAIL]${NC} $tbl tablosu hala mevcut"
97     fi
98 done
```

10. Yükseltme Sonrası Uyumluluk Testleri

Yükseltme tamamlandıktan sonra 5 kategoride toplam 28 test uygulanarak sürümün doğruluğu kanıtlanır.

10.1 Test Grubu 1: Tablo Varlık Kontrolü (15 test)

Her sürüme ait tablolar, mevcut sürüme göre dinamik olarak kontrol edilir:

Sürüm	Kontrol Edilen Tablolar
v1.0	schema_version, yazarlar, kategoriler, kitaplar, uyeler, odunc_islemleri
v2.0	yayinevleri, kitap_yazarlar, cezalar, rezervasyonlar
v3.0	dijital_kitaplar, etkinlikler, etkinlik_katilim, degerlendirmeler, bildirimler

```
51 for tbl in schema_version yazarlar kategoriler kitaplar uyeler odunc_islemleri; do
52     EXISTS=$(psql -U "$DB_USER" -d "$DB_NAME" -t -A -c "
53         SELECT COUNT(*) FROM information_schema.tables WHERE table_name='$tbl' AND
54         table_schema='public';
55         " 2>/dev/null)
56     [ "$EXISTS" = "1" ] && test_r "v1.0 tablo: $tbl" 0 || test_r "v1.0 tablo: $tbl" 1
57 done
```

10.2 Test Grubu 2: Kolon Kontrolü (7 test)

v2.0 ve v3.0 ile eklenen yeni kolonların varlığı doğrulanır:

- v2.0 kolonları: kitaplar.yayinevi_id, kitaplar.dil, uyeler.uyelik_tipi, uyeler.max_odunc
- v3.0 kolonları: kitaplar.ort_puan, kitaplar.dijital_mevcut, uyeler.toplam_odunc

10.3 Test Grubu 3: Veri Bütünlüğü (3 test)

- Foreign key ilişkilerinin sayısı kontrol edilir
- Toplam kayıt sayısının sıfırdan büyük olduğu doğrulanır
- Tablo kayıt sayıları detaylı olarak raporlanır

10.4 Test Grubu 4: View ve Fonksiyon (3 test)

- View sayısı kontrol edilir
- v_kutuphane_istatistik view'ı çalıştırılarak doğrulanır
- v_populer_kitaplar view'ı sorgulama testiyle doğrulanır

10.5 Test Grubu 5: Performans (1 test)

JOIN sorgusu çalıştırılarak yanıt süresi ölçülür. Veritabanı boyutu raporlanır:

```
154 START_P=$(date +%s%N)
155 psql -U "$DB_USER" -d "$DB_NAME" -c "SELECT COUNT(*) FROM kitaplar k JOIN yazarlar y
    ON k.yazar_id=y.yazar_id;" > /dev/null 2>&1
156 END_P=$(date +%s%N)
157 QUERY_MS=$(( (END_P - START_P) / 1000000 ))
```

10.6 Sonuç Raporu

Sürüm: 3.0.0

Toplam: 28 | Geçti: 28 | Kaldı: 0

TÜM TESTLER BAŞARILI - Yükseltme doğrulandı!

11. Tam Demo Senaryosu

06_tam_demo.sh scripti, sürüm yönetimi sürecinin tamamını uçtan uca gösteren bir demo senaryosu sunar. Demo 7 adımdan oluşur:

Adım	İşlem	Sonuç Sürüm
1	v1.0 temel şema kurulumu + veri yükleme	v1.0.0
2	Yükseltme öncesi kontrol (7 aşamalı)	v1.0.0
3	v1.0 → v2.0 migration uygulaması	v2.0.0
4	v2.0 → v3.0 migration uygulaması	v3.0.0
5	Yükseltme sonrası kontrol (28 test)	v3.0.0
6	Rollback testi (v3.0 → v2.0 geri alma)	v2.0.0
7	Tekrar v3.0 yükseltme	v3.0.0

Bu demo senaryosu ile aşağıdaki durumlar kanıtlanmıştır:

- Migration'lar sıralı ve güvenli bir şekilde uygulanabilir
- Rollback sonrası v3.0 tabloları tamamen kaldırılır
- Rollback sonrası tekrar yükseltme yapılabilir (idempotent tasarım)
- Tüm süreç boyunca veri bütünlüğü korunur

12. Sürüm Karşılaştırma Analizi

12.1 Özellik Matrisi

Özellik	v1.0	v2.0	v3.0
Tablo sayısı	6	10	15
Tahmini kolon sayısı	~35	~55	~70
İndeks sayısı	7	14	20
View sayısı	0	2	4
Fonksiyon sayısı	0	1	2
Trigger sayısı	0	0	1
Foreign Key sayısı	5	12	17
Toplam kayıt	~2.650	~3.350	~4.460
Yayınevi desteği	X	✓	✓
Çoka-çok yazar	X	✓	✓
Ceza sistemi	X	✓	✓
Rezervasyon	X	✓	✓
Dijital kitaplar	X	X	✓
Etkinlik yönetimi	X	X	✓

Özellik	v1.0	v2.0	v3.0
Değerlendirme/puan	X	X	✓
Bildirim sistemi	X	X	✓

12.2 Veritabanı Boyut Evrimi

Her sürüm geçişinde veritabanı boyutu doğal olarak büyür. Ancak eklenen indeksler sorgu performansını korurken, yeni tablolar ve view'lar iş mantığını veritabanı katmanına taşıyarak uygulama karmaşıklığını azaltır.

13. Sonuç ve Değerlendirme

13.1 Elde Edilen Kazanımlar

Bu proje kapsamında aşağıdaki kazanımlar sağlanmıştır:

- **Schema Migration Yetkinliği:** Veritabanı şemasının kontrollü ve izlenebilir bir şekilde evrimleştirilmesi
- **Atomik DDL Kullanımı:** PostgreSQL'in transactional DDL desteği ile güvenli migration işlemleri
- **Versiyonlama Altyapısı:** schema_version tablosu ile migration geçmişinin kalıcı olarak saklanması
- **Rollback Stratejisi:** Her migration için tersine çevrilebilir rollback mekanizmalarının geliştirilmesi
- **Otomatik Test Altyapısı:** 28 testle yükseltme sonrası doğrulama
- **Otomasyon Becerileri:** scripting ile migration yönetimi, ön kontrol ve sonrası test süreçlerinin otomatikleştirilmesi

13.2 En İyi Uygulamalar (Best Practices)

Projede uygulanan endüstri standardı en iyi uygulamalar:

1. **Transaction İçinde Migration:** Her DDL/DML işlemi BEGIN...COMMIT blokları içinde çalıştırılır
2. **Yükseltme Öncesi Kontrol:** Migration uygulanmadan önce ön koşullar doğrulanır
3. **Yükseltme Öncesi Yedek:** Her migration öncesi otomatik backup alınır
4. **Sürüm Kilidi:** Migration dosyaları, hedef sürümü kontrol ederek yanlış sırada çalışmayı engeller
5. **Rollback Hazırlığı:** Her migration'ın eşleşen bir rollback dosyası bulunur
6. **Yükseltme Sonrası Test:** Migration tamamlandıktan sonra kapsamlı uyumluluk testleri çalıştırılır
7. **Denetim İzi:** Tüm migration işlemleri veritabanında kayıt altına alınır

13.3 Endüstri Karşılaştırması

Özellik	Bu Proje	Flyway	Liquibase
Sürüm izleme tablosu	✓	✓	✓
Atomik migration	✓	✓	✓
Rollback desteği	✓ (manuel)	✓	✓
Ön koşul kontrolü	✓	✓	✓
Otomatik sürüm tespiti	✓	✓	✓
Denetim kaydı	✓	✓	✓
Zincirli migration	✓	✓	✓

14. Kaynakça

1. PostgreSQL Documentation — ALTER TABLE
<https://www.postgresql.org/docs/14/sql-altertable.html>
2. PostgreSQL Documentation — CREATE TABLE
<https://www.postgresql.org/docs/14/sql-createtable.html>
3. PostgreSQL Documentation — Transactional DDL
https://wiki.postgresql.org/wiki/Transactional_DDL_in_PostgreSQL
4. PostgreSQL Documentation — System Catalogs
<https://www.postgresql.org/docs/14/catalogs.html>
5. PostgreSQL Documentation — Information Schema
<https://www.postgresql.org/docs/14/information-schema.html>
6. PostgreSQL Documentation — pg_dump
<https://www.postgresql.org/docs/14/app-pgdump.html>
7. Flyway Documentation — How Flyway Works
<https://documentation.red-gate.com/fd/how-flyway-works-184127223.html>
8. Martin Fowler — Evolutionary Database Design
<https://martinfowler.com/articles/evodb.html>